

# PBA-rGAT-Edge: Arc-Level Path-based Timing Prediction with Scalable Residual Edge-Aware Graph Attention

Fangjian Liu, Chenyang Lv, Chunyang Feng

Shenzhen Giga Design Automation Co., Ltd.

Shenzhen, China

fjliu@giga-da.com, cylv@giga-da.com, cyfeng@giga-da.com

## Abstract

Path-Based Analysis (PBA) provides accurate timing by evaluating real signal transitions along true paths, but its computational cost makes full-chip deployment impractical on modern large-scale designs. Graph-Based Analysis (GBA), while much faster, often suffers from pessimism due to worst-case propagation. To bridge this accuracy–efficiency gap, we propose **PBA-rGAT-Edge**, a scalable residual graph attention model that predicts arc-level delay and slew using edge-aware message passing. Unlike prior cell-centric formulations, PBA-rGAT-Edge operates on a pin-level timing graph and explicitly embeds both cell and interconnect arcs as edge features. A lightweight residual attention backbone, coupled with multi-level feature fusion and dual-task prediction, enables efficient training and accurate timing inference. On three benchmark test cases, the model achieved a 29× to 48× inference speedup compared to commercial STA analysis, while maintaining stable acceleration performance across diverse circuit structures, significantly enhancing timing analysis efficiency. The implementation will be open-sourced at [github.com/PBA-rGAT-Edge](https://github.com/PBA-rGAT-Edge).

## Keywords

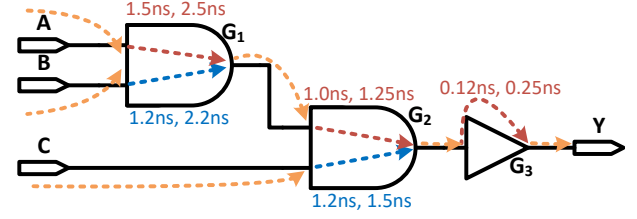
STA; PBA; Graph Neural Networks (GNN)

### ACM Reference Format:

Fangjian Liu, Chenyang Lv, Chunyang Feng. 2026. PBA-rGAT-Edge: Arc-Level Path-based Timing Prediction with Scalable Residual Edge-Aware Graph Attention. In *63rd ACM/IEEE Design Automation Conference (DAC '26)*, July 26–29, 2026, Long Beach, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770743.3810986>

## 1 Introduction

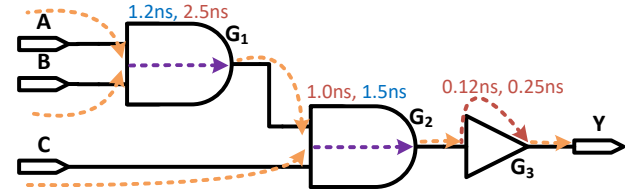
Static Timing Analysis (STA) is the backbone of timing closure in contemporary IC design, yet its complexity continues to grow as technology scaling and design sizes introduce massive timing graphs and multi-corner constraints [7]. Modern STA flows typically employ two complementary analysis modes: *Graph-Based Analysis* (GBA) and *Path-Based Analysis* (PBA) [23]. As illustrated in Fig. 1, GBA propagates worst-case arrival times across the timing graph, offering fast runtime but often introducing pessimism due to its path-agnostic nature. In contrast, PBA evaluates timing along concrete activation paths using actual transition and load conditions, thereby providing greater accuracy. However, the



$$\text{delay\_min}_{A \rightarrow Y} = 1.5\text{ns}(G_1) + 1.0\text{ns}(G_2) + 0.12\text{ns}(G_3) + \text{net\_delays}$$

$$\text{delay\_max}_{B \rightarrow Y} = 2.2\text{ns}(G_1) + 1.5\text{ns}(G_2) + 0.25\text{ns}(G_3) + \text{net\_delays}$$

(a) PBA evaluates delays along explicit timing paths, preserving actual slew and load conditions.



$$\text{delay\_min}_{A \rightarrow Y} = 1.2\text{ns}(G_1) + 1.0\text{ns}(G_2) + 0.12\text{ns}(G_3) + \text{net\_delays}$$

$$\text{delay\_max}_{B \rightarrow Y} = 2.5\text{ns}(G_1) + 1.5\text{ns}(G_2) + 0.25\text{ns}(G_3) + \text{net\_delays}$$

(b) GBA propagates worst-case delays across nodes, which can lead to pessimistic timing margins.

**Figure 1: Comparison between Path-Based Analysis (PBA) and Graph-Based Analysis (GBA).**

computational cost of PBA grows rapidly with circuit scale and path count, making full PBA sign-off infeasible for many large SoC designs [6].

To reduce the runtime overhead of PBA while preserving accuracy, recent research has explored machine-learning-based timing prediction [3, 10, 12], particularly using Graph Neural Networks (GNNs) [8, 13, 14, 19, 21, 24, 25]. GNN-based models leverage the structural correspondence between circuit netlists and timing graphs and have demonstrated strong performance across tasks such as pre-routing timing prediction [8, 21, 24], gate arrival time prediction [19], and interconnect modeling [14]. However, most of these approaches operate in a *cell-centric* manner, predicting arrival times at pins by aggregating neighborhood node embeddings. This formulation overlooks the fact that **timing arcs**, rather than nodes, are the fundamental carriers of delay, load, and slew propagation in STA [11]. Recent models have begun incorporating arc-level information. Kahng *et al.* [10] introduced learning-based correction of GBA-to-PBA timing differences at the stage level; RAPTA [3] modeled path timing using recurrent networks; and DeepEdgeGAT [22] incorporated edge-aware attention to improve



This work is licensed under a Creative Commons Attribution 4.0 International License. *DAC '26, Long Beach, CA, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2254-7/2026/07

<https://doi.org/10.1145/3770743.3810986>

PBA delay prediction. Additionally, GTN-Path [15] demonstrated that transformer-based modeling of timing paths can achieve high waveform fidelity. However, these approaches still encounter one or more limitations: (1) cell-centric message passing that weakens arc-level expressiveness; (2) deep or multi-branch architectures that slow down convergence; and (3) scalability issues that make efficient training challenging on large industrial timing graphs.

**Objective.** This work aims to design a timing prediction model that (i) directly models arc-level delay propagation, (ii) maintains architectural simplicity to ensure efficient training and inference, and (iii) achieves high accuracy consistent with PBA results. In this work, we propose **PBA-rGAT-Edge**, a residual edge-aware graph attention model tailored for path-based timing prediction. The key contributions are summarized as follows:

- **Arc-Level Timing Formulation.** Our work provides the first statistical evidence confirming systematic discrepancies in both Net Arcs and Cell Arcs between GBA and PBA modes. Building on this finding, our model innovatively adopts the timing arc as the fundamental unit for delay prediction, aiming to reduce the inherent errors across different analysis methodologies from the ground up.
- **Lightweight Residual Edge-Aware Architecture.** Tailored to the physical characteristics of STA circuits, our model introduces key enhancements to the classic GAT architecture. Specifically, it aggregates features from both neighboring nodes and connecting edges during central node updating, and incorporates residual connections to mitigate information loss and preserve crucial circuit prior information. This integrated design enables the model to efficiently capture real circuit timing behavior, leading to accelerated convergence and robust performance.
- **Performance Verification and Benefits.** Our model demonstrates superior inference accuracy, outperforming baseline models including GAT, GraphSAGE, EGNN, and DeepEdge-GAT. Furthermore, it achieves a significant improvement in inference efficiency, enabling near-real-time, PBA-level timing verification within industrial design flows and greatly accelerating chip timing closure.

## 2 Background

### 2.1 Static Timing Analysis Fundamentals

Static Timing Analysis (STA) evaluates circuit timing without dynamic simulation by propagating delays on a *timing graph*. Let the timing graph be  $G = (V, E)$ , where each node  $v \in V$  denotes a pin/port and each directed edge  $(u, v) \in E$  denotes a timing arc (cell arc or interconnect arc). Each edge carries a delay  $d(u, v)$  and electrical attributes that influence slew propagation. The arrival time (AT) at node  $v$  is computed as  $A(v) = \max_{u \in \text{pred}(v)} \{A(u) + d(u, v)\}$ , where  $\text{pred}(v)$  is the fan-in set of  $v$ .

STA primarily uses two modes: Graph-Based Analysis (GBA) and Path-Based Analysis (PBA). While GBA offers fast approximation, its deviation from PBA is non-trivial. As shown in Fig. 2, the GBA–PBA path-delay differences exhibit a clear long tail, meaning that a subset of real timing paths experiences unaffordable misestimation. Such errors can propagate into over-margining, unnecessary buffering, and reduced design performance [18]. This

### Path GBA vs PBA Delay Error Distribution

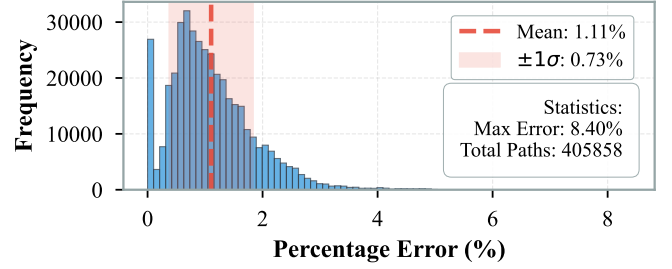


Figure 2: Path delay statistic of Design-1.

### PBA Delay Composition: Net vs Cell Delay

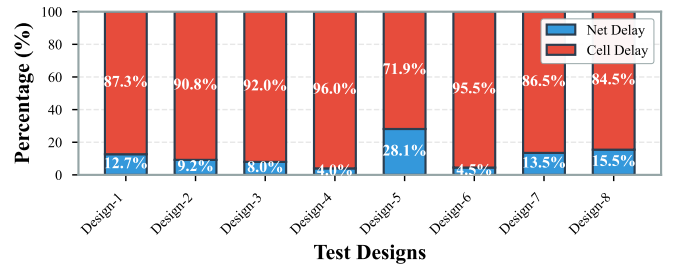


Figure 3: PBA Delay Composition.

challenge worsens at advanced technology nodes, where interconnect RC delay becomes comparable to or even exceeds cell delay [4]. Fig. 3 confirms this trend: although cell delay dominates in many designs, net delay can rise sharply (e.g., 28.1% in Design-5), making inaccurate net-level modeling a direct source of path-delay divergence. These observations highlight that accurate timing prediction requires arc-level modeling of both cell and net delays.

### 2.2 Timing Arc Decomposition and Features

Given that both cell delay and net delay contribute non-negligibly to overall timing variation (Fig. 3), our formulation explicitly models each timing arc—cell or net—as a directed edge in a pin-level graph. A timing path is a chain of such arcs. For a short segment  $net_1 \rightarrow PinA \rightarrow PinB \rightarrow net_2$ , the total delay decomposes as  $D_{\text{path}} = D_{\text{net1}} + D_{\text{cell}(A \rightarrow B)} + D_{\text{net2}}$ . We therefore construct a pin-based graph  $G = (V, E)$ , where each node  $v_i \in V$  represents a pin and each directed edge  $(p, q) \in E$  represents the timing arc driving pin  $q$ . Node features  $x_i \in \mathbb{R}^F$  encode pin-level electrical attributes (e.g., input slew type, transition limits), while edge features  $e_{pq} \in \mathbb{R}^H$  encode arc-level properties (e.g., cell-arc type, wire RC, capacitive load). Throughout the paper, “edge” and “timing arc” are used interchangeably.

### 2.3 Graph Neural Networks for Timing Graphs

Conventional neural architectures such as multi-layer perceptrons (MLPs) are inadequate for modeling the graph-structured dependencies inherent in STA. Graph Neural Networks (GNNs) [25] address this limitation through iterative message passing. GAT [20] computes content-dependent attention coefficients to weight neighbor messages, enabling adaptive importance assignment across incoming arcs. GraphSAGE [9] aggregates sampled neighborhood

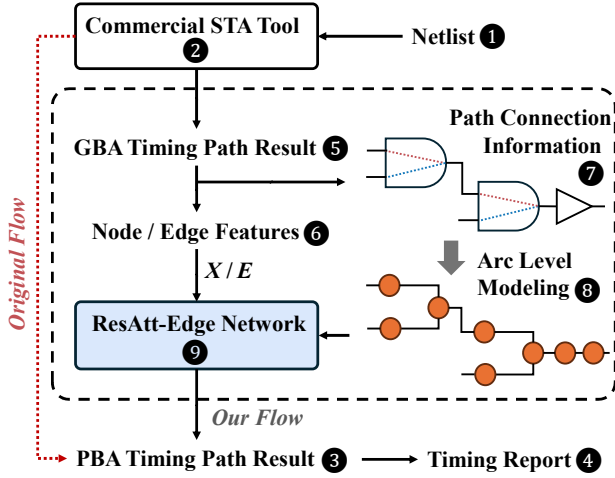


Figure 4: Overall workflow with PBA-rGAT-Edge.

information using mean/pooling operators, supporting scalable inductive learning on large timing graphs. EGNN [5] extends standard GNNs by incorporating multi-dimensional edge features through doubly-stochastic normalization and layer-wise edge adaptation, improving tasks where edge attributes play a critical role.

### 3 Motivation and Problem Formulation

#### 3.1 Motivation

Accurate arc-level delay prediction is essential for timing closure, where even small errors may trigger false violations or unnecessary optimization iterations [2]; However, conventional GNN models face fundamental limitations in achieving this precision: MLP, GAT [20], and GraphSAGE [9] models exclusively utilize node features as input, discarding edge features. This leads to non-convergence during training on industrial-scale cases. EGNN and DeepEdgeGAT models aggregate only node features during central node aggregation, ignoring edge attributes and layout information. Consequently, they exhibit slower convergence compared to edge-aware architectures. Furthermore, DeepEdgeGAT exclusively predicts Cell Arc delays, while neglecting Net Arc delay errors. This omission compromises path delay accuracy in timing analysis tasks. In GAT-based models, aggregated features are propagated as node features in subsequent layers, causing gradual dilution of intrinsic node characteristics. This undermines training stability and feature expressiveness.

Therefore, we propose the PBA-rGAT-Edge model, which performs delay prediction based on timing arcs to address the computational discrepancies between Net Arcs and Cell Arcs under GBA and PBA methodologies. Specifically, when aggregating information for a central node, our model incorporates both neighborhood node features and edge features. This design is grounded in the circuit characteristics that the transition time of the preceding output pin determines the input slew of the current cell, while the load capacitance from the subsequent input pin directly influences the output delay; furthermore, attributes of the downstream timing arc indirectly alter the load conditions by affecting parasitic parameters of the interconnect network. To preserve the pessimistic boundary

characteristics from the GBA analysis as a benchmark for PBA correction, thereby ensuring stability during the optimization process, we introduce residual connections to the aggregated node features.

As shown in Fig. 4, starting from the circuit’s input netlist (1), a conventional industrial PBA flow (1–4) obtains accurate timing reports by running full PBA within commercial STA tools (2–3), which is computationally expensive. In contrast, the faster GBA mode (2–5) can produce approximate timing paths at much lower cost. Leveraging this efficiency, our framework begins with GBA timing-path results (5) and extracts corresponding node and edge features  $X/E$  (6) as well as arc-level connection relationships (7–8). These are then fed into the proposed residual edge-aware attention network (9), which predicts PBA-quality arc delays and slews without re-running full PBA, thereby reducing the overall runtime to generate the final timing report (4).

#### 3.2 Problem Definition

Motivated by the accuracy and scalability limitations of existing graph-based timing predictors discussed above, we formalize the *timing arc prediction problem*. In this formulation, a timing path is represented as an ordered sequence of timing arcs  $\mathcal{P} = \{a_1, a_2, \dots, a_K\}$ , where each arc  $a$  corresponds to either a cell arc or an interconnect segment, each associated with a propagation delay  $\hat{D}_a$ . The predicted path delay is therefore

$$\hat{T}_{\text{path}} = \sum_{a \in \mathcal{P}} \hat{D}_a, \quad (1)$$

where  $\hat{D}_a$  is produced by a learned arc-level model that maps local electrical and topological features (e.g., input slew, output capacitance, wire  $R/C$ ) to delay estimates. The objective is to obtain per-arc predictions whose aggregated delay  $\hat{T}_{\text{path}}$  closely matches the ground-truth PBA value from commercial STA tools, enabling accurate PBA-quality timing without running PBA.

### 4 PBA-rGAT-Edge Framework

#### 4.1 Overview

As shown in Fig. 5 summarizes the three-stage workflow of PBA-rGAT-Edge. (1) **Feature and graph construction.** From GBA timing-path reports, we extract pin-level electrical features, arc-level cell/net attributes, and construct a pin-based timing graph whose directed edges correspond to timing arcs. This representation enables fine-grained arc-level modeling beyond conventional gate-centric GNNs. (2) **Residual edge-aware attention layers.** A stack of ResAtt-Edge layers performs edge-aware message passing, where node features and arc attributes jointly determine attention coefficients. Each layer applies residual updates to stabilize optimization and expand the effective receptive field. All intermediate node embeddings are retained for multi-scale fusion. (3) **Arc-level prediction head.** Multi-level node embeddings are concatenated and fed into two lightweight MLP heads to predict arc slew and delay, providing PBA-quality estimates without invoking full PBA. This pipeline captures both local electrical effects at arc boundaries and long-range path interactions, enabling accurate and scalable timing prediction for large industrial designs.

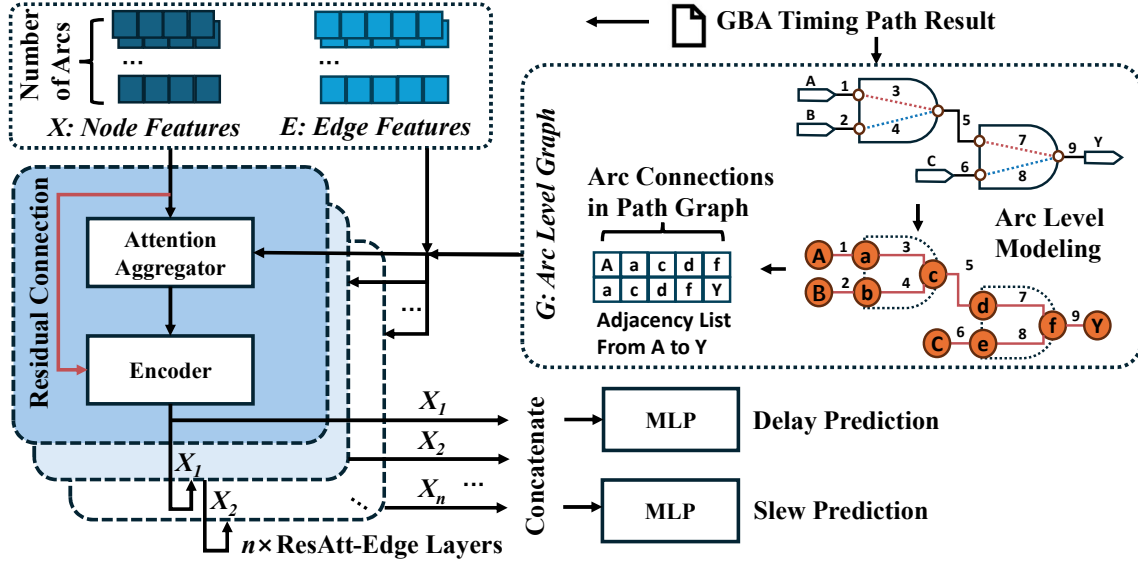


Figure 5: Overview of the PBA-rGAT-Edge framework.

## 4.2 Feature and Graph Construction

PBA-rGAT-Edge adopts a *pin-level graph* with *arc-level edge features*, enabling direct modeling of both cell and net delays. Each pin is treated as a graph node, and each timing arc ( $p \rightarrow q$ ) is represented as a directed edge carrying its electrical attributes. **Node and edge attributes.** Each pin is assigned a node feature vector, and each timing arc (cell or net arc) is assigned an edge feature vector, with all feature definitions summarized in Table 1. These form the node feature matrix  $X$  and edge feature matrix  $E$ . **Graph construction.** From the GBA timing-path report, we extract timing paths and build their corresponding pin-level graphs. Although paths may overlap, each is expanded to include not only the arcs on the main chain but also arcs sharing fanout with it, capturing local cross-branch timing dependencies. Arc connectivity is encoded using a directed adjacency list in the form of a PyTorch Geometric edge index  $\text{edge\_index} \in \mathbb{N}^{2 \times \text{num\_edges}}$ , where each column represents a directed pin-to-pin dependency. **Final representation.** The triplet  $(X, E, G)$ —pin features, arc features, and the constructed pin-level timing graph—feeds into the Residual Edge-Aware Attention layers described next. This representation aligns the graph structure with the learning target: predicting arc-level delay and slew via per-pin embeddings.

## 4.3 Residual Edge-Aware Attention Layer

Each ResAtt-Edge layer refines arc-level representations using two operations: an edge-aware attention aggregation, and a residual encoding block. This design enables deep message passing while maintaining gradient stability and preserving arc-level electrical information.

**Edge-Aware Attention Aggregation.** Given the layer-wise node features  $\mathbf{x}_i^{d-1} \in \mathbb{R}^{D_n}$  and the edge features  $\mathbf{e}_{ij} \in \mathbb{R}^{D_e}$  (which remain fixed across layers), the edge-aware attention aggregator performs message passing on the arc-level timing graph  $G = (V, E)$ . For each

directed arc  $(j \rightarrow i) \in E$ , we first compute an attention score by jointly considering the target node, the source node, and the arc attributes:

$$s_{ij}^d = f_{\text{att}}^{(d)} \left( [\mathbf{x}_i^{d-1} \parallel \mathbf{x}_j^{d-1} \parallel \mathbf{e}_{ij}] \right), \quad (2)$$

where  $f_{\text{att}}^{(d)}(\cdot)$  is a lightweight two-layer MLP with LeakyReLU activation, and  $[\cdot \parallel \cdot]$  denotes feature concatenation. The normalized attention coefficient  $\alpha_{ij}^d$  is obtained via a softmax over the incoming neighbors of node  $i$ :

$$\alpha_{ij}^d = \frac{\exp(s_{ij}^d)}{\sum_{k \in \mathcal{N}(i)} \exp(s_{ik}^d)}, \quad j \in \mathcal{N}(i). \quad (3)$$

The edge-aware message from node  $j$  to node  $i$  is computed as  $\mathbf{m}_{ij}^d = \alpha_{ij}^d [\mathbf{x}_j^{d-1} \parallel \mathbf{e}_{ij}]$ , and all incoming messages are aggregated by  $\mathbf{g}_i^d = \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^d$ . Thus,  $\mathbf{g}_i^d$  jointly captures logical context (via node features) and electrical arc characteristics (via edge features), and is subsequently fed into the residual encoding block.

**Residual Encoding.** Given the aggregated representation  $\mathbf{g}_i^d$ , the encoder block performs dimension reduction followed by residual fusion with the layer input features. Specifically, the encoder applies a linear projection, LayerNorm, and a LeakyReLU activation:

$$\tilde{\mathbf{x}}_i^d = \sigma \left( \text{LN} \left( \mathbf{W}_{\text{enc}}^{(d)} \mathbf{g}_i^d + \mathbf{b}_{\text{enc}}^{(d)} \right) \right), \quad \mathbf{W}_{\text{enc}}^{(d)} \in \mathbb{R}^{D_n \times (2D_n + D_e)}. \quad (4)$$

This reduces the concatenated message vector back into the node feature space. The final node representation is obtained through a residual shortcut:

$$\mathbf{x}_i^d = \mathbf{x}_i^{d-1} + \tilde{\mathbf{x}}_i^d, \quad \forall i \in V. \quad (5)$$

This formulation allows each ResAtt-Edge layer to learn task-dependent corrections over the original timing-arc representation while maintaining an information-preserving highway across layers. In practice, such residual connections mitigate optimization difficulties

and over-smoothing in deep GNNs, leading to more stable training and higher accuracy.

#### 4.4 Multi-Level Feature Fusion and Dual-Head Prediction

After passing through  $D$  stacked ResAtt-Edge layers, each node  $i$  obtains a sequence of hierarchical representations  $\{\mathbf{x}_i^1, \dots, \mathbf{x}_i^D\}$ , where deeper layers encode progressively larger arc-level receptive fields. The merge module concatenates these multi-level features into a unified fused embedding:

$$\mathbf{f}_i = \parallel_{d=1}^D \mathbf{x}_i^d, \quad \mathbf{f}_i \in \mathbb{R}^{D \cdot D_n}, \quad \forall i \in V. \quad (6)$$

This simple concatenation preserves both shallow local context and deep global path information, and serves as the input to the downstream prediction head. This simple concatenation preserves both shallow local context and deep global path information, and serves as the input to the downstream prediction head.

Although the prediction targets (slew and delay) are defined on timing arcs, our graph formulation uses pins as nodes. In sign-off STA reports, each arc ( $j \rightarrow i$ ) attaches its slew/delay value to the destination pin  $i$ . Consequently, the per-pin embedding  $\mathbf{f}_i$  naturally serves as the arc-level representation for the arc terminating at pin  $i$ . The dual-head predictor therefore performs arc-level timing regression directly over pin embeddings.

On top of the fused representation  $\mathbf{f}_i$ , PBA-rGAT-Edge adopts a lightweight dual-head MLP predictor for simultaneous slew and delay regression. Concretely, two independent two-layer perceptrons map the fused features to the predicted slew and delay:

$$S_p(i) = \text{MLP}_{\text{slew}}(\mathbf{f}_i), \quad D_p(i) = \text{MLP}_{\text{delay}}(\mathbf{f}_i), \quad (7)$$

where each MLP consists of a linear layer, ReLU activation, and a final linear projection to a scalar output. Let  $S_r(i)$  and  $D_r(i)$  denote the reference PBA slew and delay obtained from sign-off timing analysis. The training objective is a weighted sum of Huber losses:

$$\mathcal{L} = \beta \cdot \text{HuberLoss}(S_p, S_r) + (1-\beta) \cdot \text{HuberLoss}(D_p, D_r), \quad (8)$$

where  $\beta \in (0, 1)$  controls the relative emphasis between slew and delay. The final path delay is obtained by accumulating the timing arc delays along the same path.

#### 4.5 Training and Inference

For each design in Fig. 7a, we extract PBA timing paths and construct the corresponding arc-level graphs as described in Sec. 4.2. PBA-rGAT-Edge is trained end-to-end using the dual-head regression loss in Eq. equation 8, where Huber loss is adopted to reduce sensitivity to outlier PBA values. We optimize the model using Adam [1] and employ a ReduceLROnPlateau scheduler [17] to automatically decay the learning rate once validation performance saturates. Early stopping [16] is further applied to avoid overfitting and unnecessary epochs.

### 5 Experimental Results

The experiments were performed on a high-performance server with four Intel Xeon Platinum 8356H CPUs, 1 TB DDR4 ECC RAM, and an NVIDIA RTX 4080 GPU, running CentOS 7.6. As summarized in Fig. 7a, our dataset is constructed from the worst-case setup timing paths reported by the commercial STA tool. Hold

---

#### Algorithm 1 PBA-rGAT-Edge Training Procedure

---

**Input:** Arc-level graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, H)$ ; node features  $X = \{x_i \mid i \in \mathcal{V}\}$ ; edge features  $H = \{e_{ij} \mid (i, j) \in \mathcal{E}\}$ ; PBA labels  $S_r, D_r$ ; depth  $D=5$ .  
**Output:** Trainable params  $W = \{W_X^d\}_{d=1}^D$  in ResAtt-Edge layers, and MLP params  $\theta, \phi$ .

- 1: **for**  $d = 1$  to  $D$  **do**
- 2:   **for**  $i \in \mathcal{V}$  **do**
- 3:     Compute  $\alpha_{ij}^d$  via Eq. equation 3,  $j \in \mathcal{N}(i)$
- 4:      $g_i^d \leftarrow \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^d (x_j^d \parallel e_{ij}^d)$      *Edge-aware aggregation*
- 5:     Update  $x_i^d$  via Eq. equation 5     *Residual encoding*
- 6:   **end for**
- 7: **end for**
- 8: **for**  $i \in \mathcal{V}$  **do**
- 9:    $f_i \leftarrow \text{Concat}(x_i^1, \dots, x_i^D)$      *Multi-level feature fusion*
- 10:    $S_p^i \leftarrow \text{MLP}_\theta(f_i)$      *Predict arc slew*
- 11:    $D_p^i \leftarrow \text{MLP}_\phi(f_i)$      *Predict arc delay*
- 12: **end for**
- 13: Compute loss  $\mathcal{L}$  via Eq. equation 8
- 14: Update all parameters  $W, \theta, \phi$  using Adam [1]

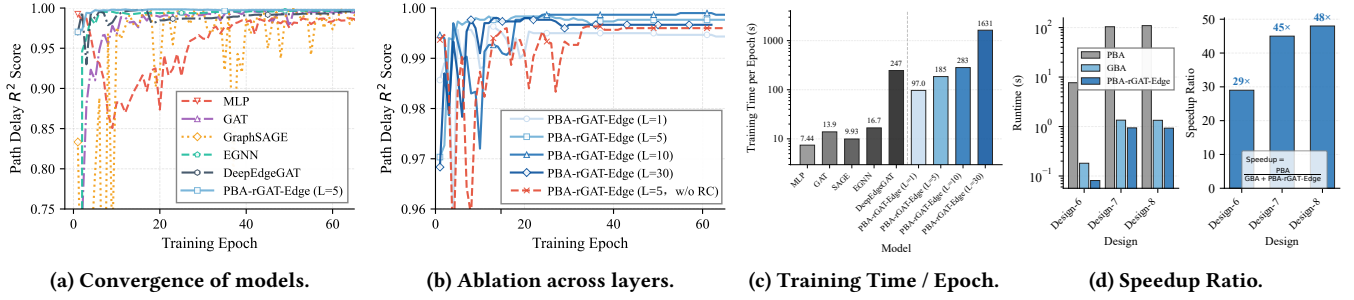
---

**Table 1: Definition of Node&Edge Attributes**

| Type        | Name                     | Description                             |
|-------------|--------------------------|-----------------------------------------|
| <b>Node</b> | slew_type                | Transition type of the pin (rise/fall). |
|             | max_rise_transition_time | Max rise transition time of the pin.    |
|             | max_fall_transition_time | Max fall transition time of the pin.    |
|             | direction_of_signal      | Pin direction (input or output).        |
| <b>Edge</b> | GBA_delay                | Delay of the timing arc from GBA.       |
|             | arc_type                 | Type of the arc (cell arc or net arc).  |
|             | max_total_resistance     | Maximum total resistance on the arc.    |
|             | ccs_total_max_rise_cap   | Max capacitive load for rising edge.    |
|             | ccs_total_max_fall_cap   | Max capacitive load for falling edge.   |

paths were excluded due to their nearly identical analysis flow and feature processing compared to setup paths. Including both would introduce redundancy without enhancing modeling performance. PBA-rGAT-Edge operates on timing arcs, where edge features encode cell/net arc attributes and node features capture the electrical properties of the pins connected by the edges. The original implementations of MLP, GAT, and GraphSAGE were designed to process node features exclusively, lacking native support for edge feature integration. When trained solely on node features, these models exhibited non-convergence due to their inability to capture critical structural dependencies encoded in edge attributes. To resolve this limitation, we augmented the input representation by concatenating edge features to the destination node features, thereby enabling joint modeling of node and edge information. Since the source code of the DeepEdgeGAT model was not publicly available, we independently implemented its architecture and evaluated six variants with 1, 5, 10, 25, 50, and 100 layers on our test set. While the slew prediction performance across all configurations was suboptimal, the best-performing variant (selected based on convergence stability and metric balance) is summarized in Table 2.



Figure 6: Convergence behavior and ablation studies of PBA-rGAT-Edge on path-delay  $R^2$  metrics.Table 2: Unified Comparison of Arc-Level (Slew/Delay  $R^2$ ) and Path-Level ( $R^2$ /MAE) Timing Prediction

| Design   | #Arcs   | Arc-Level Prediction ( $R^2$ Score) |       |           |       |       |       |       |       |             |       |               |              | Path Delay Prediction ( $R^2$ /MAE(ps)) |              |              |             |              |                    |
|----------|---------|-------------------------------------|-------|-----------|-------|-------|-------|-------|-------|-------------|-------|---------------|--------------|-----------------------------------------|--------------|--------------|-------------|--------------|--------------------|
|          |         | MLP                                 |       | GraphSAGE |       | GAT   |       | EGNN  |       | DeepEdgeGAT |       | PBA-rGAT-Edge |              | MLP                                     | GraphSAGE    | GAT          | EGNN        | DeepEdgeGAT  | PBA-rGAT-Edge      |
|          |         | Slew                                | Delay | Slew      | Delay | Slew  | Delay | Slew  | Delay | Slew        | Delay | Slew          | Delay        |                                         |              |              |             |              |                    |
|          |         | $(R^2/MAE)$                         |       |           |       |       |       |       |       |             |       |               |              | $(R^2/MAE)$                             |              |              |             |              |                    |
| Design-6 | 82,886  | 0.942                               | 0.986 | 0.906     | 0.993 | 0.943 | 0.995 | 0.939 | 0.997 | 0.818       | 0.995 | <b>0.948</b>  | <b>0.998</b> | 0.967/32.334                            | 0.992/20.236 | 0.997/11.364 | 0.998/8.478 | 0.994/15.406 | <b>0.999/3.568</b> |
| Design-7 | 375,844 | 0.940                               | 0.988 | 0.961     | 0.997 | 0.984 | 0.997 | 0.985 | 0.997 | 0.828       | 0.997 | <b>0.985</b>  | <b>0.998</b> | 0.996/6.923                             | 0.987/15.435 | 0.994/9.696  | 0.997/6.830 | 0.998/5.369  | <b>0.999/3.277</b> |
| Design-8 | 679,824 | 0.956                               | 0.988 | 0.956     | 0.997 | 0.989 | 0.998 | 0.984 | 0.997 | 0.889       | 0.995 | <b>0.989</b>  | <b>0.999</b> | 0.995/7.136                             | 0.997/15.411 | 0.994/8.101  | 0.997/5.257 | 0.995/6.145  | <b>0.998/4.271</b> |
| Average  | 379,518 | 0.946                               | 0.987 | 0.941     | 0.996 | 0.972 | 0.997 | 0.969 | 0.997 | 0.845       | 0.996 | <b>0.974</b>  | <b>0.998</b> | 0.986/15.464                            | 0.992/17.027 | 0.995/9.720  | 0.997/6.855 | 0.996/8.973  | 0.999/3.705        |

| Set      | Design   | #Pins      | #Nets      |
|----------|----------|------------|------------|
| Training | Design-1 | 8,200,026  | 8,078,268  |
|          | Design-2 | 2,559,486  | 2,455,568  |
|          | Design-3 | 163,226    | 150,874    |
|          | Design-4 | 376,540    | 366,506    |
|          | Design-5 | 19,064,180 | 18,377,535 |
| Test     | Design-6 | 82,886     | 81,370     |
|          | Design-7 | 725,938    | 709,332    |
|          | Design-8 | 684,548    | 667,942    |

(a)

(b)

Table 3: Training Epochs and Runtime Across Models

| Metric         | MLP | GraphSAGE | GAT    | EGNN  | DeepEdgeGAT | PBA-rGAT-Edge |
|----------------|-----|-----------|--------|-------|-------------|---------------|
| Epochs         | 75  | 235       | 802    | 197   | 183         | 9             |
| Train time (s) | 558 | 2,333     | 11,124 | 3,292 | 45,116      | 1,669         |

significantly faster convergence than other variants, highlighting the critical role of both layer depth and residual connections in

Figure 7: Benchmark statistics (left) and ablation about aggregate mechanism (right).

## 5.1 Performance Comparison

As shown in Table 3, the model achieved convergence after only 9 training epochs, demonstrating the fastest convergence speed among all evaluated models. This indicates that the model is highly optimized for STA circuits. Furthermore, it achieved exceptional inference accuracy on unseen designs, with a slew  $R^2$  score of 0.974 and a delay  $R^2$  score of 0.998. Notably, the path delay prediction performance was outstanding, reaching an  $R^2$  score of 0.999 and a maximum MAE (Mean Absolute Error) of just 4.2 ps. As shown in Fig. 6d, the model achieves a remarkable 29× to 45× speedup compared to the PBA mode of a commercial STA tool, highlighting its superior computational efficiency for industrial-scale applications. As illustrated in Fig. 6a and Table 3, the PBA-rGAT-Edge model achieves the fastest convergence, requiring the fewest epochs among all tested models, while its total training time is second only to the MLP baseline.

## 5.2 Ablation Study

As illustrated in Figs. 6b and 6c, we first conducted an ablation study to evaluate the impact of different architectural configurations on training convergence. Specifically, we compared models with 1, 5, 10, and 30 GAT layers, along with a variant where residual connections were removed. The 5-layer GAT configuration achieved

Table 3: Training Epochs and Runtime Across Models

| Metric         | MLP | GraphSAGE | GAT    | EGNN  | DeepEdgeGAT | PBA-rGAT-Edge |
|----------------|-----|-----------|--------|-------|-------------|---------------|
| Epochs         | 75  | 235       | 802    | 197   | 183         | 9             |
| Train time (s) | 558 | 2,333     | 11,124 | 3,292 | 45,116      | 1,669         |

significantly faster convergence than other variants, highlighting the critical role of both layer depth and residual connections in optimizing training dynamics. The model exhibited a significant performance degradation in training after removing residual connections, highlighting their critical role in maintaining gradient flow and feature refinement capabilities. Our ablation study on the central aggregation mechanism reveals that removing edge features during aggregation leads to training non-convergence, as illustrated in Fig. 7b. Both compared models employ a 5-layer architecture. The results demonstrate that simultaneously aggregating node and edge features during central node aggregation enables the model to learn STA circuit characteristics more effectively, resulting in faster convergence.

## 6 Conclusion

In this work, we identified inherent discrepancies in the delays of both net arcs and cell arcs when computed under GBA versus PBA modes. Motivated by this finding, we developed a novel GAT model that predicts delays directly from timing arcs. By incorporating residual connections and aggregating features from both neighboring nodes and connecting edges during central node updates, our model effectively captures the underlying physics of STA timing paths. This approach not only significantly reduces training difficulty but also achieves high predictive accuracy on unseen designs. Moreover, it provides substantial speedup over commercial STA tools in PBA computation, paving the way for practical industrial deployment.

## References

- [1] Kingma DP Ba J Adam et al. 2014. A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* 1412, 6 (2014).
- [2] Lilas Alrahis et al. 2022. GNN4REL: Graph neural networks for predicting circuit reliability degradation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 3826–3837.
- [3] Tanmoy Chowdhury, Ashkan Vakil, Banafsheh Saber Latibari, Seyed Aresh Beheshti Shirazi, Ali Mirzaeian, Xiaojie Guo, Sai Manoj PD, Houman Homayoun, Ioannis Savidis, Liang Zhao, et al. 2022. RAPTA: A hierarchical representation learning solution for real-time prediction of path-based static timing analysis. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 493–500.
- [4] Jeffery A Davis et al. 2001. Interconnect limits on gigascale integration (GSI) in the 21st century. *Proc. IEEE* 89, 3 (2001), 305–324.
- [5] Xiaojie Gong and Kaizhu Cheng. 2019. Exploiting Edge Features for Graph Neural Networks. In *CVPR*.
- [6] Guannan Guo, Tsung-Wei Huang, Yibo Lin, and Martin Wong. 2021. Gpu-accelerated path-based timing analysis. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 721–726.
- [7] Zizheng Guo, Tsung-Wei Huang, Zhou Jin, Cheng Zhuo, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. Heterogeneous static timing analysis with advanced delay calculator. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [8] Zizheng Guo, Mingjie Liu, Jiaqi Gu, Shuhan Zhang, David Z Pan, and Yibo Lin. 2022. A timing engine inspired graph neural network model for pre-routing slack prediction. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 1207–1212.
- [9] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [10] Andrew B Kahng, Uday Mallappa, and Lawrence Saul. 2018. Using machine learning to predict path-based slack from graph-based timing analysis. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*. IEEE, 603–612.
- [11] Pei-Yu Lee and Iris Hui-Ru Jiang. 2018. iTimerM: A compact and accurate timing macro model for efficient hierarchical timing analysis. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 23, 4 (2018), 1–21.
- [12] Rongjian Liang, Zhiyao Xie, Jinwook Jung, Vishnavi Chauha, Yiran Chen, Jiang Hu, Hua Xiang, and Gi-Joon Nam. 2020. Routing-free crosstalk prediction. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
- [13] Zihao Lin, Haisen Zhang, Peng Gao, Fei Yu, Tingting Wu, Xiaoming Xiong, and Shuting Cai. 2025. GNN-Based Timing Prediction in Pre-Routing Stage With Multi-Task Learning Strategy. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [14] Fangzhou Liu, Guannan Guo, Yuyang Ye, Ziyi Wang, Wenjie Fu, Weihua Sheng, and Bei Yu. 2025. GraphCAD: Leveraging Graph Neural Networks for Accuracy Prediction Handling Crosstalk-affected Delays. In *Proceedings of the 2025 International Symposium on Physical Design*. 125–133.
- [15] Lihao Liu, Beisi Lu, Yunhui Li, Li Shang, and Fan Yang. 2025. GTN-Path: Efficient Path Timing Prediction through Waveform Propagation with Graph Transformer. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [16] Lutz Prechelt. 2002. Early stopping-but when? In *Neural Networks: Tricks of the trade*. Springer, 55–69.
- [17] PyTorch. [n. d.]. PyTorch: ReduceLROnPlateau. PyTorch 1.9.0 documentation, [https://pytorch.org/docs/stable/generated/torch.optim.lr\\_scheduler.ReduceLROnPlateau.html#torch.optim.lr\\_scheduler.ReduceLROnPlateau](https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html#torch.optim.lr_scheduler.ReduceLROnPlateau).
- [18] Sneha Saurabh, Hitarth Shah, and Shivendra Singh. 2019. Timing closure problem: Review of challenges at advanced process nodes and solutions. *IETE Technical Review* (2019).
- [19] Pratik Shrestha, Saran Phatharodom, and Ioannis Savidis. 2022. Graph representation learning for gate arrival time prediction. In *Proceedings of the 2022 ACM/IEEE Workshop on Machine Learning for CAD*. 127–133.
- [20] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [21] Ziyi Wang, Siting Liu, Yuan Pu, Song Chen, Tsung-Yi Ho, and Bei Yu. 2023. Restructure-tolerant timing prediction via multimodal fusion. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [22] Yuyang Ye, Tinghuan Chen, Yifei Gao, Hao Yan, Bei Yu, and Longxing Shi. 2023. Graph-learning-driven path-based timing analysis results predictor from graph-based timing analysis. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 547–552.
- [23] Zuodong Zhang, Zizheng Guo, Yibo Lin, Runsheng Wang, and Ru Huang. 2022. Eventtimer: Fast and accurate event-based dynamic timing analysis. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 945–950.
- [24] Ruizhe Zhong, Junjie Ye, Zhentao Tang, Shixiong Kai, Mingxuan Yuan, Jianye Hao, and Junchi Yan. 2024. Preroutggnn for timing prediction with order preserving partition: Global circuit pre-training, local delay learning and attentional cell modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17087–17095.
- [25] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI open* 1 (2020), 57–81.